

QuizForge - System Architecture Documentation

Project Overview

QuizForge is a full-stack quiz management platform designed to support secure online quiz conduction, automated evaluation, leaderboard tracking, and role-based access control.

The platform supports two primary user roles:

- ✧ **Admin**
 - Creates and manages quizzes
 - Evaluates results
 - Controls visibility and quiz operations
- ✧ **Student**
 - Attempts quizzes
 - Views results and leaderboard
 - Accesses quiz history

The system is built using a modern JavaScript full-stack architecture with:

- ✧ **Frontend:** React.js (Vite + JSX)
- ✧ **Backend:** Node.js + Express.js
- ✧ **Database:** Firebase Firestore
- ✧ **Authentication:** JWT + bcrypt

1. High-Level Architecture

The application follows a layered client-server architecture.

User → React Frontend → Express API Server → Services Layer → Firebase Firestore

The architecture is divided into:

1. Frontend Layer

2. API & Routing Layer
3. Middleware Layer
4. Controller Layer
5. Service Layer
6. Database Layer

2. Frontend Architecture

Technology Stack

- ✧ React.js
- ✧ Vite
- ✧ JSX
- ✧ Context API
- ✧ REST API Integration

Responsibilities

The frontend handles:

- ✧ User authentication
- ✧ Quiz participation
- ✧ Admin dashboard interactions
- ✧ API communication
- ✧ State management
- ✧ UI rendering

Core Components

UI Components

Responsible for:

- ✧ Pages
- ✧ Forms
- ✧ Quiz interfaces
- ✧ Navigation

- ⌘ Dashboard rendering

AuthContext

Manages:

- ⌘ User session
- ⌘ Authentication state
- ⌘ JWT persistence
- ⌘ Role-based UI rendering

Quiz API Service

Handles:

- ⌘ REST API requests
- ⌘ Token attachment
- ⌘ Quiz fetch operations
- ⌘ Result submissions

3. Backend Architecture

The backend is built using **Node.js** and **Express.js** following modular architecture principles.

Routing Layer

Handles all API endpoints.

Major Routes

Route	Purpose
/api/auth	Authentication
/api/admin	Admin operations
/api/student	Student operations
/api/quiz	Quiz management

4. Middleware Layer

Middleware is used for security, validation, and access control.

Authentication Middleware

JWT Validation

Responsible for:

- ⌘ Verifying JWT tokens
- ⌘ Authenticating users
- ⌘ Attaching user data to requests

Role Middleware

Provides:

- ⌘ Admin-only route protection
- ⌘ Student-specific access restrictions

Timer Middleware

Validates:

- ⌘ Quiz timing constraints
- ⌘ Submission windows
- ⌘ Attempt expiration

5. Controller Layer

Controllers handle incoming HTTP requests and delegate business logic to services.

Auth Controller

Handles:

- ⌘ Registration
- ⌘ Login
- ⌘ Token generation

- ✧ Session handling

Quiz Controller

Responsible for:

- ✧ Quiz CRUD operations
- ✧ Quiz retrieval
- ✧ Quiz visibility handling

Student Controller

Handles:

- ✧ Quiz attempts
- ✧ Fetching quizzes
- ✧ Result viewing

Admin Controller

Responsible for:

- ✧ Quiz management
- ✧ Result evaluation
- ✧ Administrative operations

6. Service Layer

The service layer contains core business logic.

Auth Service

Features:

- ✧ User registration
- ✧ Password hashing
- ✧ JWT token creation
- ✧ Credential verification

Uses:

- ✧ bcrypt
- ✧ JWT
- ✧ Firebase SDK

Quiz Service

Handles:

- ✧ Quiz CRUD operations
- ✧ Question management

Student Service

Responsible for:

- ✧ Quiz attempts
- ✧ Result storage
- ✧ Performance tracking

Admin Service

Handles:

- ✧ Evaluation logic
 - ✧ Leaderboard management
 - ✧ Administrative analytics
 - ✧ Quiz Publishing and Unpublishing
-

7. Database Architecture

Firebase Firestore

Firestore is used as the primary NoSQL database.

Collections

Users Collection

Stores:

Given in the Repository QuizForge/backend/models/ folder.

8. Authentication Workflow

Login Process

1. User submits credentials
2. Auth Controller receives request
3. Auth Service validates credentials
4. bcrypt compares passwords
5. JWT token is generated
6. Token returned to frontend
7. Frontend stores token
8. Token attached to future API calls

9. Quiz Attempt Workflow

Student Quiz Flow

1. Student logs in
2. Frontend fetches available quizzes
3. Student starts quiz
4. Timer middleware validates duration
5. Answers submitted to backend
6. Student Service processes submission
7. Results stored in Firestore
8. Leaderboard updated

10. Admin Workflow

Quiz Management Flow

1. Admin logs in
2. Admin dashboard loads
3. Admin creates/edits quizzes
4. Quiz Service performs CRUD operations
5. Firestore updates quiz collection
6. Updated quizzes become visible to students

11. Security Features

JWT Authentication

- ✧ Stateless authentication
- ✧ Secure route protection
- ✧ Token validation middleware

Password Security

- ✧ Password hashing using bcrypt
- ✧ Plain-text passwords never stored

Role-Based Access Control

- ✧ Separate admin/student permissions
- ✧ Protected routes

Middleware Validation

- ✧ Timer validation (Not implemented Yet)
- ✧ Authentication checks
- ✧ Authorization enforcement

12. Scalability Considerations

The architecture is designed for scalability through:

- ✧ Modular service architecture
- ✧ Separation of concerns
- ✧ Stateless JWT authentication
- ✧ Firestore cloud scalability
- ✧ Independent frontend/backend deployment

Potential future improvements:

- ✧ Redis caching
- ✧ WebSockets for real-time leaderboards
- ✧ Docker containerization
- ✧ CI/CD pipelines
- ✧ Microservices architecture

13. Performance Optimizations

Implemented optimizations include:

- ✧ Efficient REST API design
- ✧ Optimized Firestore queries
- ✧ Lightweight frontend rendering
- ✧ Token-based authentication
- ✧ Modular backend services

14. Future Enhancements

Potential upgrades:

- ✧ Real-time quiz monitoring
- ✧ AI-based question generation
- ✧ Analytics dashboard

- ✧ Email notifications
- ✧ Multiplayer quiz mode
- ✧ Live contest support

Github link:- <https://github.com/BhavesGadling77/QuizForge>

Live link:- <https://quiz-forge-robn.vercel.app/>